

Recobra - Reto de corte 1 (linea base / stack obligatorio)

Equipo: Camilo Conde, Fernando Conde, Miguel Jacome, Veronica Ubarne

Repo: https://github.com/ISCOUTB/AS_202620_Recobra

ADR del reto: docs/adr/0003-reto-corte1-stack-obligatorio.md

Medicion: docs/medicion-corte1.md

1. Diagnostico de la restriccion

El curso fija un espacio cerrado de tecnologias: backend NestJS o FastAPI; frontend Flutter o NextJS. No se asigno otra restriccion de calidad aparte del stack. Impacto: el backend previo en Express quedaba fuera del espacio permitido; el C4 ya nombraba Flutter pero no habia cliente en el repositorio. El escenario ancla es S5 (mantenibilidad): migrar el framework sin reescribir el dominio ni romper el corte vertical.

2. Alternativas y decision

A) NestJS + Flutter - experiencia previa del equipo en Node/Flutter; Nest materializa puertos/adaptadores con DI; Flutter encaja con uso en campus (camara/push).

B) FastAPI + NextJS - OpenAPI util, pero obliga a Python y a un cliente web sin experiencia previa fuerte del equipo.

C) Mantener Express - costo cero, pero incumple el requisito del curso.

Decision: NestJS + Flutter, conservando arquitectura hexagonal (ADR-0002 / ADR-0003).

3. Cambio aplicado sobre el corte vertical

Composition root y adaptador HTTP migrados a NestJS (src/main.ts, publicaciones/, salud/). Dominio y casos de uso en TypeScript dependen solo del puerto PublicacionRepository (persistencia en memoria reemplazable). Cliente Flutter en mobile/ crea y consulta publicaciones via REST. CI en .github/workflows/ci.yml ejecuta npm test, e2e y flutter test. Limites C4 conservados: API + cliente; PostgreSQL/auth/notificaciones siguen como objetivo.

4. Línea base, resultado y contraste con el umbral

Línea base (Express hexagonal previo): pruebas del vertical slice en verde; contenedor API fuera del stack permitido; cliente móvil ausente; latencia POST /publicaciones en orden de 8-15 ms en local.

Resultado (NestJS + Flutter, 2026-09-05):

- npm test y npm run test:e2e en verde (dominio + corte e2e).
- flutter test en verde.
- Contenedor API = NestJS (permitido); cliente Flutter presente.
- Latencia POST /publicaciones (N=50, script npm run measure:post): p50=0.67 ms, p95=2.02 ms (objetivo local <=100 ms).
- Reglas de Publicación intactas (S5 cumplido: cambio de framework sin reescribir dominio).

Procedimiento reproducible: npm install && npm run start; en otra terminal npm run measure:post. Detalle en docs/medicion-corte1.md.

5. Trazabilidad (aspecto -> requisito -> C4 -> ADR -> código -> pruebas -> evidencia)

Aspecto A2 (docs/aspectos.md) -> escenario S5 -> contenedor API NestJS / app Flutter (docs/c4/README.md) -> ADR-0002 y ADR-0003 -> src/domain, src/application, src/publicaciones, mobile/ -> npm test, npm run test:e2e, flutter test, CI GitHub Actions -> docs/medicion-corte1.md y este PDF.

Uso de IA del corte registrado en docs/ia.md (aceptado ADR/CI/Flutter; rechazado FastAPI/NextJS).

Etiqueta git esperada: corte-1 sobre el commit de esta entrega. Sustentación: la restricción fue solo el stack obligatorio; la medición demuestra S5 y latencia del POST del corte vertical.